A Qualified Replacement for `#pragma once`

Document: P0538R0
Date: 2016-10-27
Project: ISO/IEC JTC1 SC22 WG21 Programming Language C++
Audience: Evolution Working Group
Author: Matthew Woehlke (mwoehlke.floss@gmail.com)

Abstract

This proposal recommends to standardize `#once` as an improved mechanism for preventing multiple inclusion of a header, and a related directive `#forget`.

Contents

Problem

It is well known that when compiling code of non-trivial complexity, the complete set of `#include` directives may reference the same header more than once. This often occurs when a translation unit uses several distinct components which each rely on the same base component (especially library configuration headers, headers that provide export decoration symbols, and the like). While this is correct for each component header in order to allow it to be used on its own, the combination of multiple components requires a mechanism to prevent the definitions in a header from being parsed twice, which would lead to compile errors.

Traditionally, this is accomplished with "include guards", which take the form:

```
// foo.h
#ifndef _MYLIB_FOO_H_INCLUDED
#define _MYLIB_FOO_H_INCLUDED
...
#endif // _MYLIB_FOO_H_INCLUDED
```

At least one problem with this is obvious; the guard symbol is repeated as many as three times (the last occurrence in the comment is optional and at least has no impact on compiling if it is incorrect), leading to the possibility of mistakes when retyping the symbol that cause the guard to be ineffective. Less obvious, but even more problematic, it is common for headers to be copied, which can lead to difficult to diagnose errors if the programmer neglects to adjust the guard when doing so.

Some compilers support `#pragma once` as an alternate mechanism for preventing multiple inclusions. However, many problems with this mechanism are known. It is difficult for compiler authors to implement correctly, especially in the presence of pathological source trees (involving copies of headers, whether by symlink, or worse, the same physical file accessible via

different mount points). There is also a question of how distinct headers providing similar definitions should be handled. These problems are well addressed by traditional include guards.

Proposal

We propose to introduce three new preprocessor directives in an attempt to address this issue.

`#once`

> **#once** *identifier* [<*whitespace*> *version*]

The *identifier* shall consist of one or more C++ identifiers (sequences of alphanumeric characters and/or _, not starting with a digit) joined by `::` (henceforth referred to as a "qualified name"). The *version*, if specified, shall be a token string consisting of alphanumeric characters and/or the _ or `.` characters, or a string literal, and shall set the version associated with the specified *identifier*.

If a previous `#once` directive having the same *identifier* and *version* has been previously seen, the compiler shall ignore the remainder of the `#include` unit. If the *identifier* is known but the *version* does not match, the program shall be ill-formed. (If *version* is unspecified, the version shall be the empty string.)

`#forget`

> **#forget** *identifier*

The compiler shall remove the *identifier* from its collection of previously seen identifiers. This directive provides a mechanism to force the multiple inclusion of an `#include` unit which uses `#once`.

Comments and Examples

Static Analysis

As mentioned, one of the problems with traditional guards is that they can easily get out of sync with the header file they guard. While it is possible to write static analysis tools to detect such errors, the proliferation of different styles of guards make it difficult to write a single heuristic that works across a broad base of existing software. In turn, this means that such tools tend to be project specific and are at best run when code is committed to a repository. It would be far better for such checks to be integrated into the compiler, so that they run at build time, and can be promoted to errors.

We address this by making the guard identifier a qualified name. Besides being more consistent with C++ conventions (for example, the namespace of the guard could match the namespace of the project which owns the header), this, combined with the introduction of a new feature, makes it straight forward to stipulate that the unqualified portion of the identifier shall match the name of the `#include` unit (excluding a file extension, if any).

Moreover, it is not inconceivable that we could agree that the namespace portion of the qualified identifier shall match the namespace of the definitions provided by the `#include` unit (so that all parts of the guard identifier are checked for correctness), with the compiler issuing a diagnostic if the `#include` unit does not include at least one declaration in the same namespace.

Since we are talking about QoI issues here, we feel that it is not necessary that these checks be normative. Instead, we would prefer to let the compiler community agree on what conventions should be expected and diagnosed.

Example

```
// foo.h
#once MyLibrary::bar // warning: guard should be 'MyLibrary::foo'

// bar.h
#once bar // warning: guard should be namespaced
```

Proper Use of Versioning

Although the "obvious" way to use version directives is to include the version of the software package to which a header belongs in every single header, this leads to an obvious and significant maintenance burden. A better solution which will be equally adequate in almost every instance is to maintain such version information in a single, global header file (e.g. `version.h`, `config.h`,

`exports.h`) which is always included via an `#include` directive (prior to `#once`) whose path is marked with quotes (`""`) rather than angle brackets (`<>`). This ensures that the global header is always found in a known location relative to the header being processed, and will in almost all cases be sufficient to catch mismatching versions of the header which includes the global header.

Another option, which can be employed in tandem, is to use a monotonically increasing version number that is unique to each header and is incremented whenever the interface(s) defined in the header change. Because this number is unique to the header, and only changes when the header changes (and possibly not even that frequently), the maintenance burden is significantly reduced.

The relatively liberal specification of allowed version strings was chosen with the specific intention of encouraging the version string to be generated by the build system, and in particular to allow the version string to include a VCS identifier. In this way, we may ensure that headers from a development version of software are not mixed with those from a release version or different development version, even if the normative version number does not differ between such versions.

Example

```
// version.h
#once MyLibrary::version 0.1.0 // MyLibrary version 0.1.0

// widget.h
#include "version.h"
#once MyLibrary::widget 2 // widget API version 2

// common.h
#include "version.h"
#once MyLibrary::common // no version
```

Performance

One of the points that is frequently raised in favor of `#pragma once` is that it allows the compiler to skip reading a file that it has already included. However, the problem with this is that if the compiler is not able to correctly determine if a header has already been included, it is likely that the translation unit will fail to compile.

In fact, compilers may and do already implement similar logic for traditional include guards. By employing a heuristic, a compiler may determine that a header's contents are entirely guarded. Having done so, the header and its guard may be entered into a map, such that the compiler may choose not to read the header a second time if it observes that an `#include` directive would reference a header that has been previously processed and whose include guard is defined. This is safer, since in case of a wrong guess, the compiler will read the header anyway and process it as empty due to the traditional guard, which has a small performance penalty but does not affect correctness of the program.

Our model for `#once` provides these same benefits, while making explicit (and enforcing) that the entire header may be skipped if the compiler "knows" it has been included already. The proposed directive therefore provides the same performance benefits as `#pragma once`, but without the potential pitfalls. (In cases such as described above, where one or more `#include` directives precede `#once`, the compiler would need to track the recursive set of guards which make a second inclusion a no-op. While somewhat more complicated, this still seems achievable.)

Discussion

Why not reuse `#pragma`?

The obvious answer is that `#pragma` as a whole is implementation defined. Choosing an entirely new directive makes it clear that this feature is "blessed" by the standard and not an implementation defined feature. The exact names used, however, are subject to the usual bikeshedding. We would encourage the committee to consider the feature first on its merits; if it seems useful, we are completely open to choosing some other name or even syntax for the directives. (It might even make sense to use a syntax that is evocative of that used by modules.)

Won't modules make this irrelevant?

It is possible that modules will significantly reduce the need for this feature, but modules aren't here yet, and it is likely that we will continue to have traditional headers for a long time. Since this feature happens entirely at the preprocessor level, it is our sincere hope that compilers will choose to implement the feature early, and enable it regardless of the language level requested. This means that existing software may be able to take advantage of the feature much sooner than such software can be ported to

modules (which will involve a much more invasive change).

Shouldn't this go to C first?

While we would certainly love to see this feature adopted by C as well, we don't think it makes sense that preprocessor features *must* be adopted by C first. In particular, we note that the use of a C++ qualified identifier gives us a very good reason to adopt this feature in C++ first, as C will have to decide to either accept C++ qualified identifiers for this purpose or find an alternate solution that solves the same problems that are addressed by the use of a qualified name.

Moreover, we note that it does not make a significant difference in practice which language adopts a preprocessor feature first. Since most compilers share preprocessor function between C and C++ front-ends, adoption of this feature by C++ will likely make it a de facto C standard.

Summary

We have shown a mechanism for implementing a next generation system for preventing multiple inclusion of headers. This system is semantically equivalent to traditional guards, and so avoids the known issues of present implementations of `#pragma once` (without an identifier). By also providing a `#forget`, we address the issue of how to force multiple inclusion when necessary in a way that does not require editing the header in question. By using a qualified identifier, we provide an improved mechanism for avoiding collisions that is also amenable to the use of static analysis tools to detect the sorts of improper use that are the major complaint against traditional guards. By also specifying an optional mechanism for providing version information, we provide a means to diagnose accidental mixing of different versions of headers.

Acknowledgments

We wish to thank Hans Guijt for complaining loudly enough about standardizing `pragma once` that we decided to actually write a proposal, Tim Song for valuable feedback on the initial draft, and everyone else on the `std-proposals` forum that contributed comments on this topic.